Clusterisation of the main conformation of proteins

Viktoria Ivanova - 7MI3400799

In this project, advanced research on the polypeptide chains will be held, as them being the main construction unit of the proteins. Their possible conformations will be investigated by clustering the phi and psi combinations of rotations, as some of them are more favourable and others are not possible to happen in the polypeptide chain at all.

Imports

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors
```

Loading the data

In [2]:
```
proteins_df = pd.read_csv("data_assignment3.csv")
proteins_df
```

Out[2]:

	residue name	position	chain	phi	psi
0	LYS	10	A	-149.312855	142.657714
1	PRO	11	A	-44.283210	136.002076
2	LYS	12	A	-119.972621	-168.705263
3	LEU	13	A	-135.317212	137.143523
4	LEU	14	A	-104.851467	95.928520
...	...	...	...	...	...
29364	GLY	374	B	-147.749557	155.223562
29365	GLN	375	B	-117.428541	133.019506
29366	ILE	376	B	-113.586448	112.091970
29367	ASN	377	B	-100.668779	-12.102821
29368	LYS	378	B	-169.951240	94.233680

29369 rows × 5 columns

In [3]:
```
proteins_df.dtypes
```

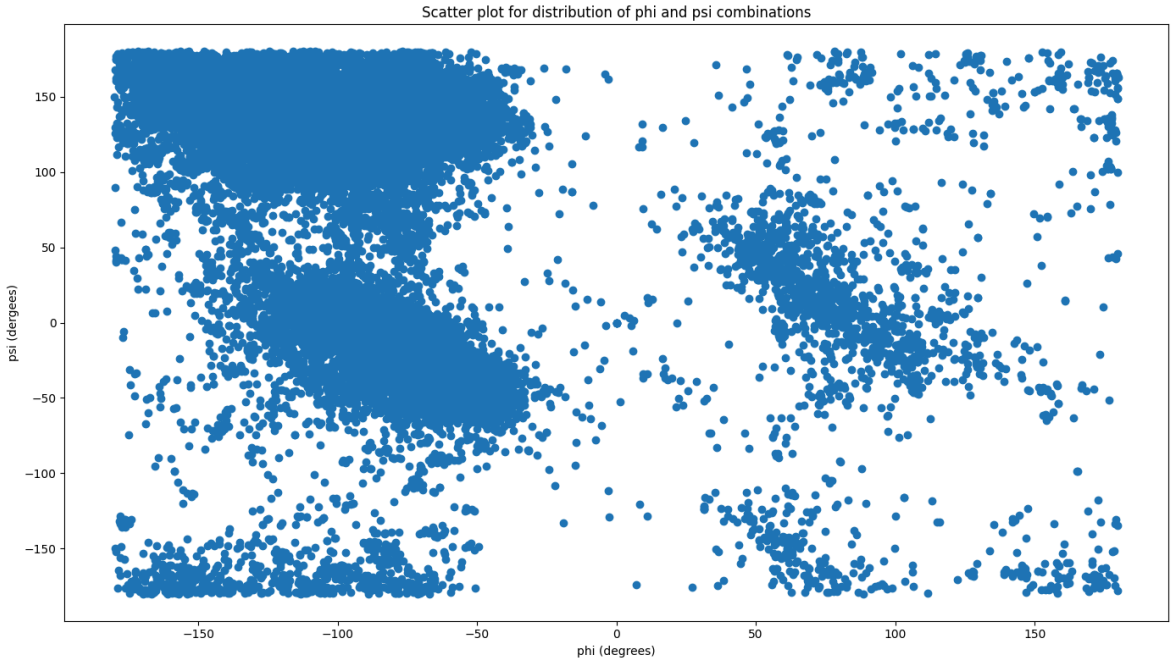
Out[3]:
```
residue name     object
position          int64
chain            object
phi             float64
psi             float64
dtype: object
```

We can conclude that there are 5 features, 2 of them are categorical - 'Residue name' and 'Chain' and the other 3 are numerical - the position of the aminoacid and the phi and psi angles, presented in degrees.

1. Show the distribution of phi and psi combinations using:

   a. A scatter plot

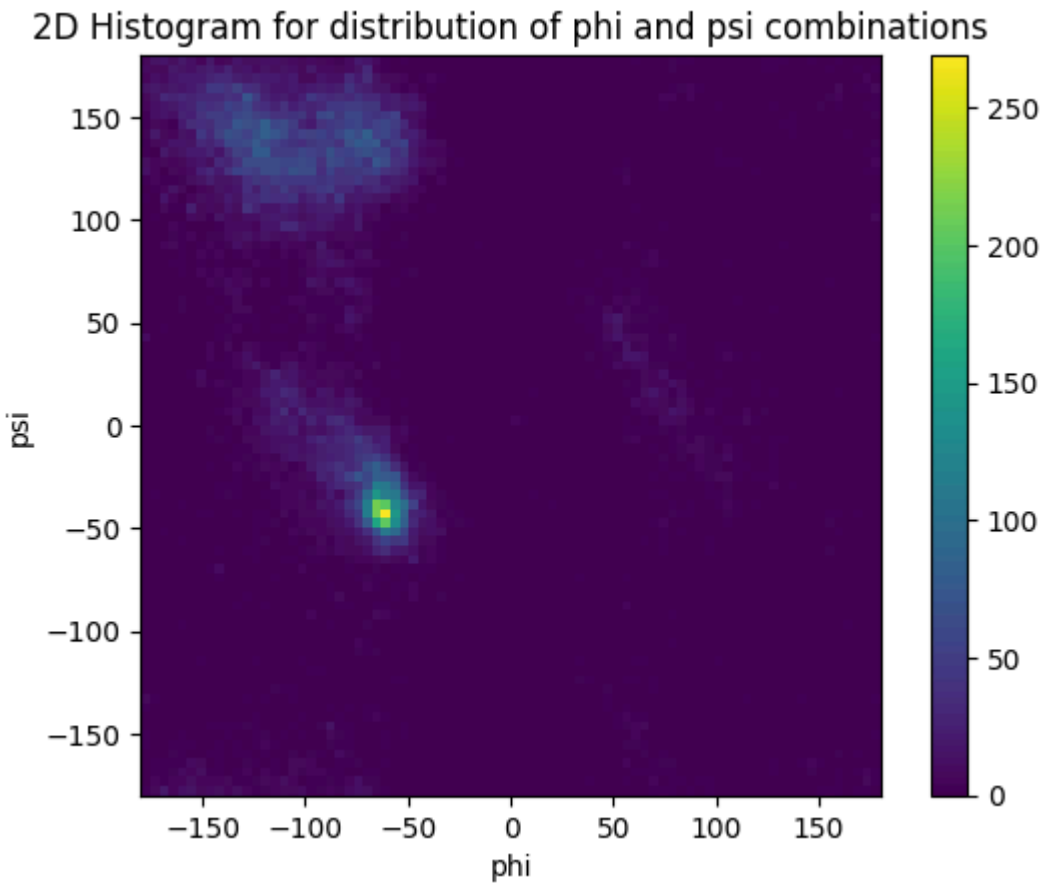
In [4]:
```
plt.figure(figsize=(14,8))
plt.scatter(proteins_df['phi'], proteins_df['psi'])
plt.xlabel(f'phi (degrees)')
plt.ylabel('psi (dergees)')
plt.title('Scatter plot for distribution of phi and psi combinations')

plt.tight_layout()
plt.show()
```



We can see 7 more divided clusters, but depending on the algorithm used, they can be less.

b. A 2D histogram

In [5]:
```
plt.hist2d(proteins_df['phi'], proteins_df['psi'], bins=80)
plt.xlabel("phi")
plt.ylabel("psi")
plt.title("2D Histogram for distribution of phi and psi combinations")
plt.colorbar()
plt.gca().set_aspect('equal')
```

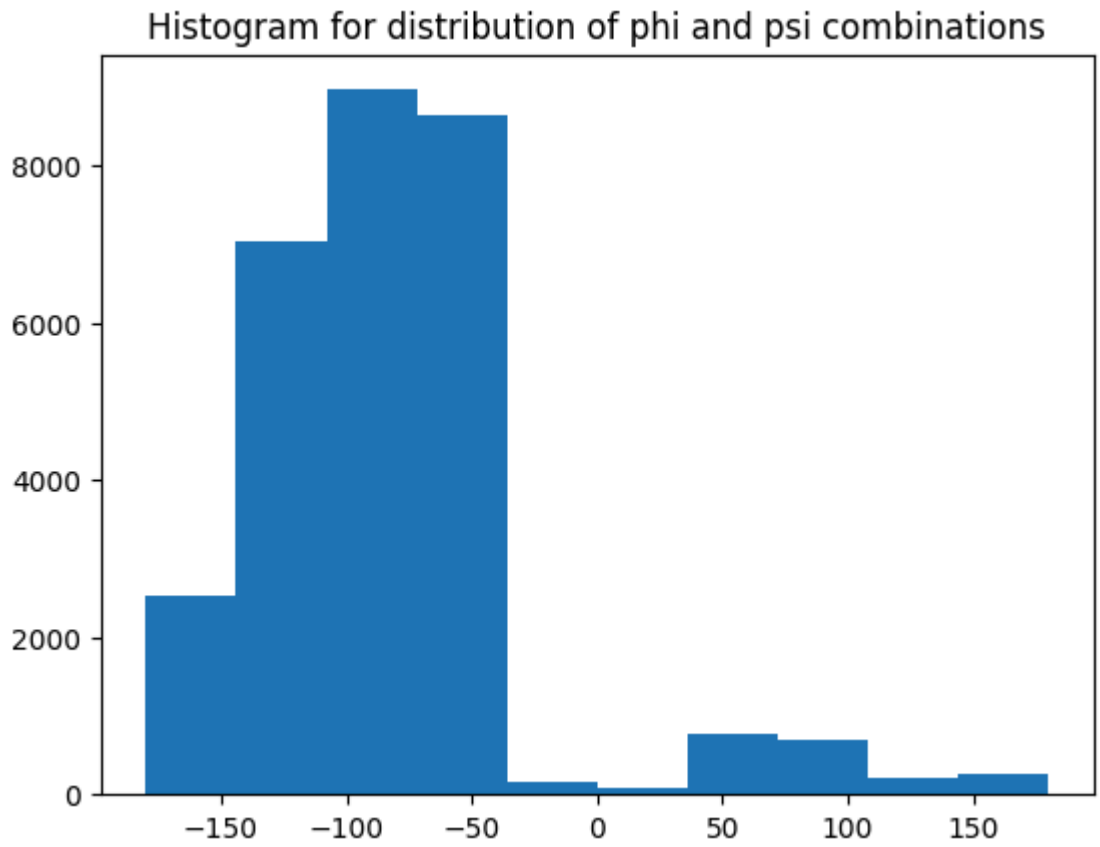


Here, only 3 clusters are outstandingly visible, although, 5 can be seen as well.

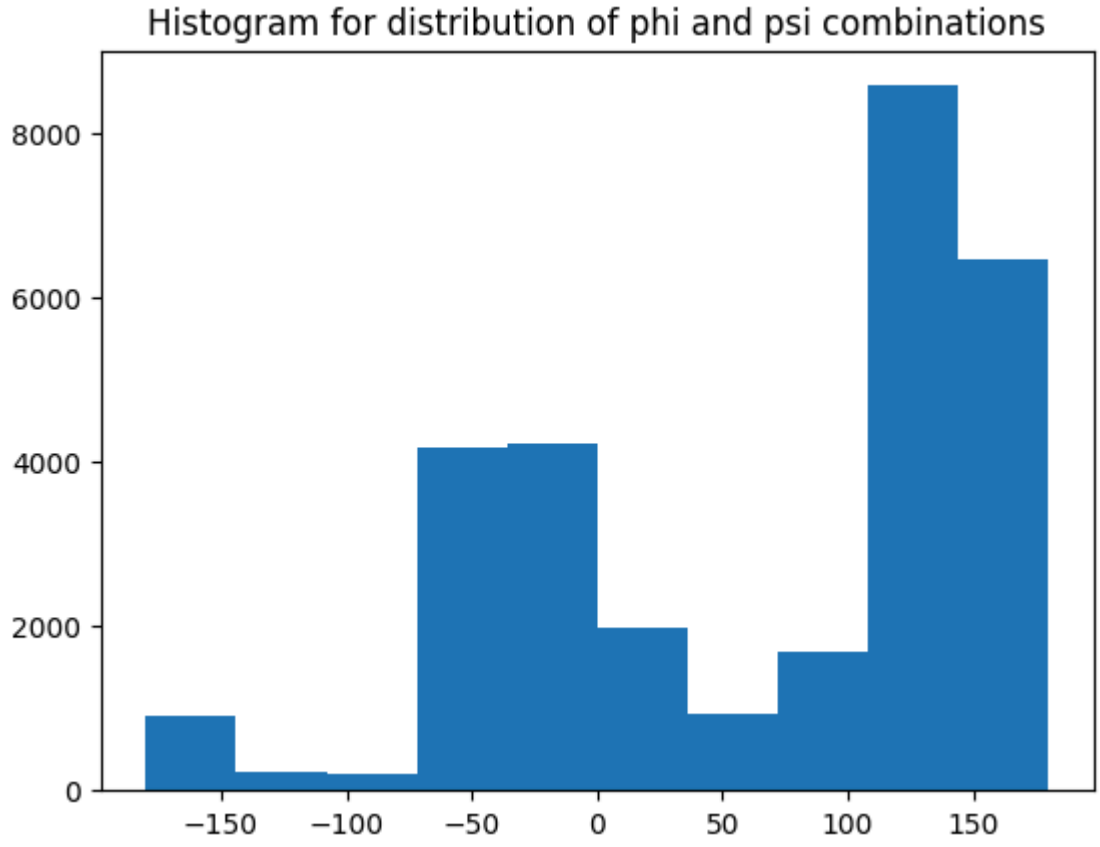
Here we can also check the distribution of separate phi and psi componenents to see outstanding number of samples for a specific diapason:

In [6]:
```
for label, data in proteins_df.items():
    if data.dtypes != 'object' and label != 'position':
        print(f'Mean value for {label} : {np.round(np.mean(data), 3)}')
        print(f'Median value for {label} : {np.round(np.median(data), 3)}')
        print(f'Standard deviation for {label} : {np.round(np.std(data), 3)}')

        plt.title("Histogram for distribution of phi and psi combinations")
        plt.hist(data, bins=10)
        plt.show()

    print()
```

```
Mean value for phi : -82.362
Median value for phi : -85.198
Standard deviation for phi : 56.847
```




```
Mean value for psi : 64.252
Median value for psi : 110.903
Standard deviation for psi : 91.118
```



2. Use the K-means clustering method to cluster the phi and psi angle combinations in the data file

In [7]:
```
clustering_proteins_df = proteins_df[['phi', 'psi']]
clustering_proteins_df
```

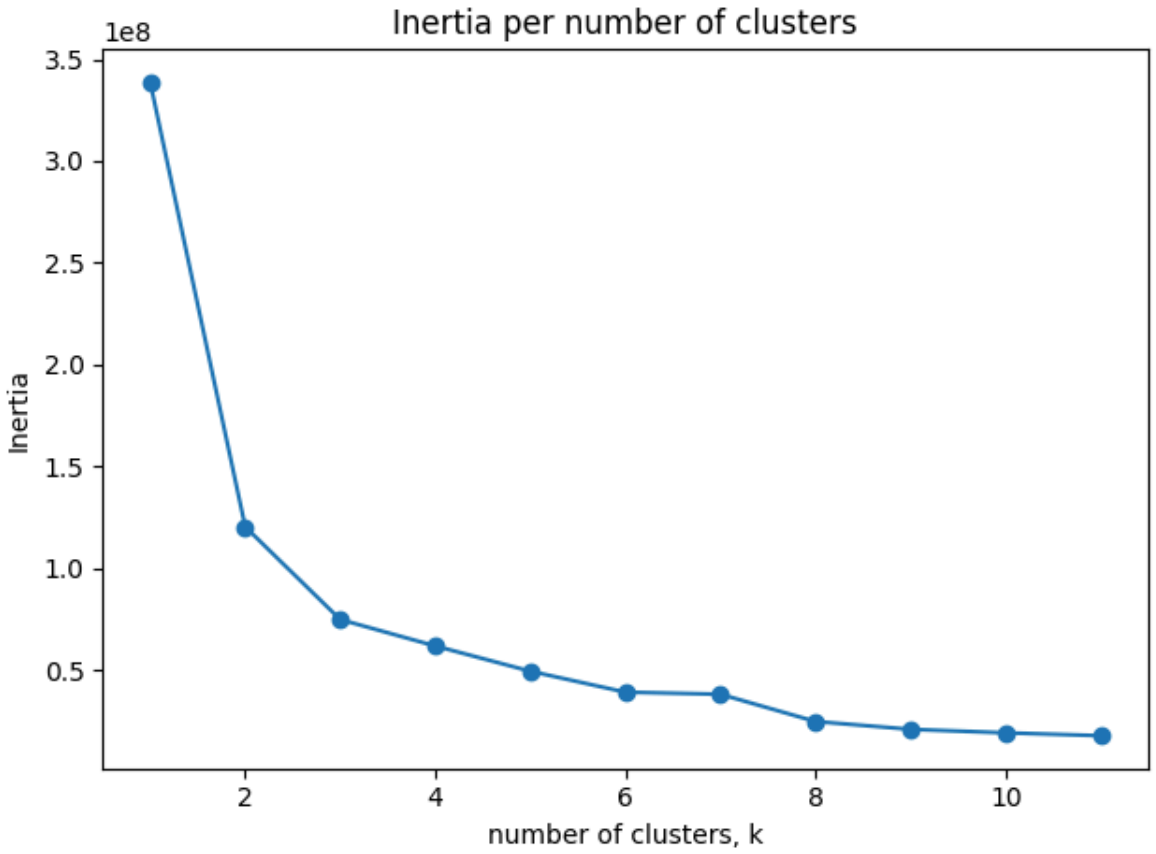
Out[7]:

	phi	psi
0	-149.312855	142.657714
1	-44.283210	136.002076
2	-119.972621	-168.705263
3	-135.317212	137.143523
4	-104.851467	95.928520
...	...	...
29364	-147.749557	155.223562
29365	-117.428541	133.019506
29366	-113.586448	112.091970
29367	-100.668779	-12.102821
29368	-169.951240	94.233680

29369 rows × 2 columns

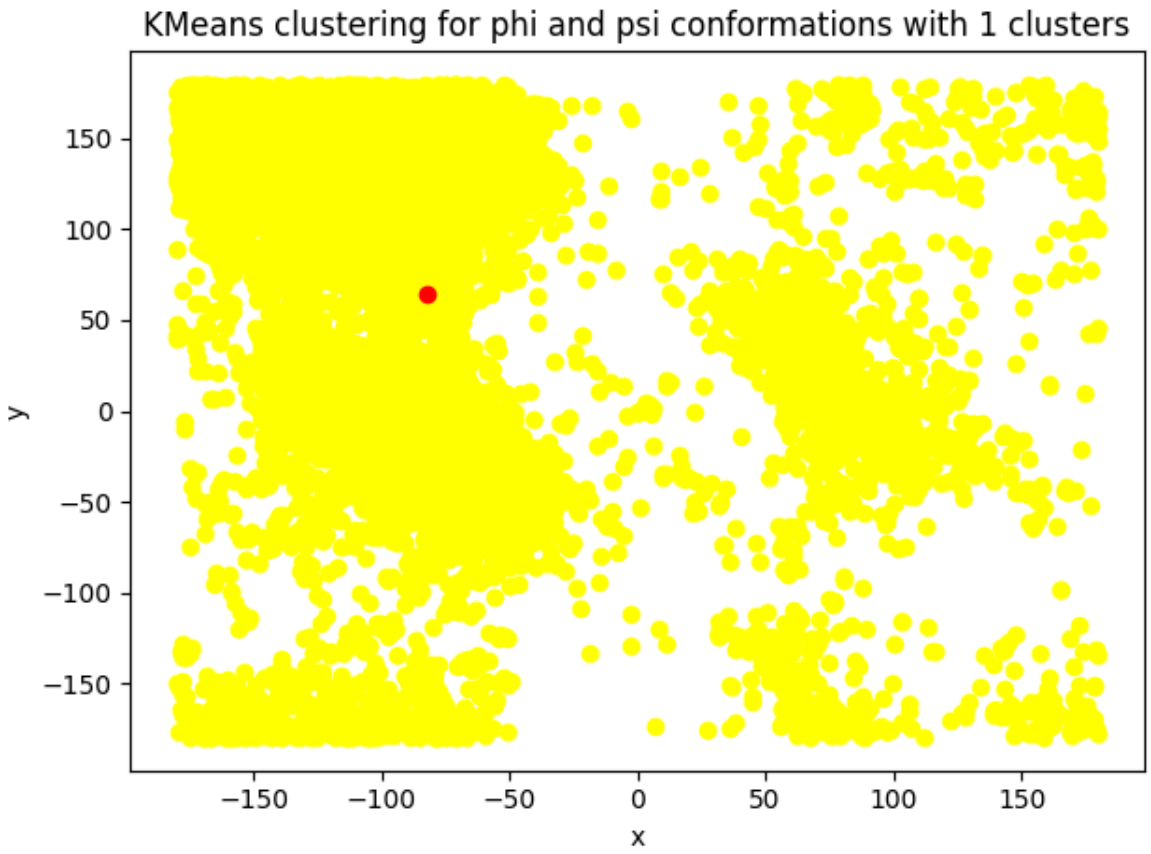
Based on the inertia, we can deside the best number of clusters.

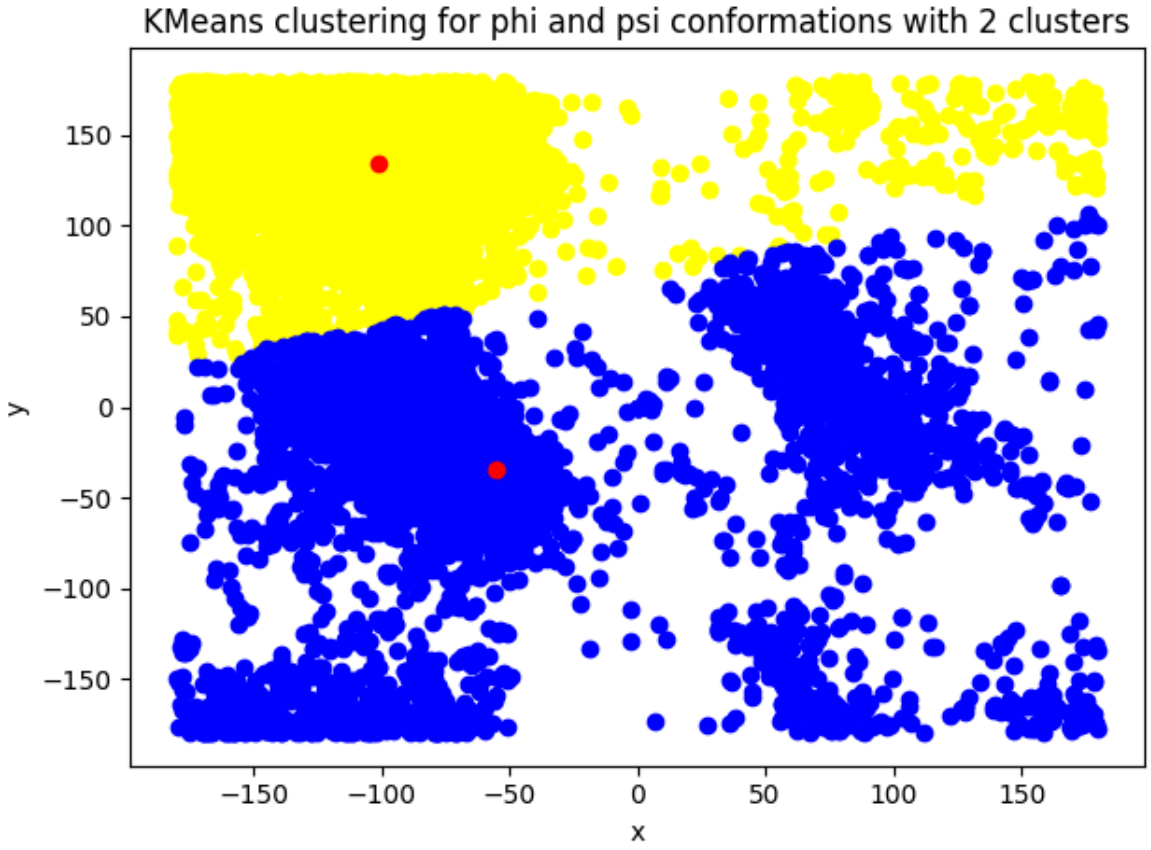
In [8]:
```
clusters = np.arange(1, 12)
inertia_list = []

for num_clusters in clusters:
    kmeans = KMeans(n_clusters=num_clusters)
    kmeans.fit(clustering_proteins_df)
    inertia_list.append(kmeans.inertia_)

plt.plot(clusters, inertia_list)
plt.scatter(clusters, inertia_list)
plt.title('Inertia per number of clusters')
plt.xlabel('number of clusters, k')
plt.ylabel('Inertia')
plt.tight_layout()
plt.show()
```

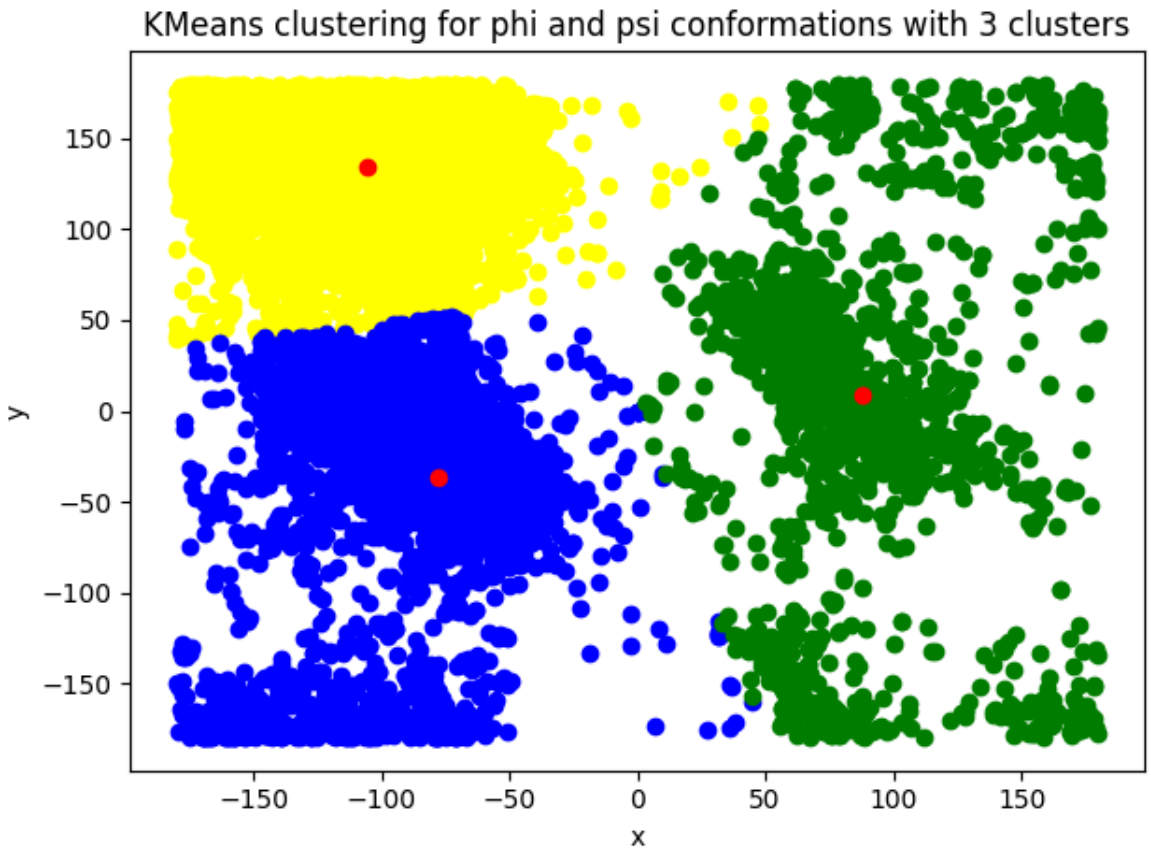


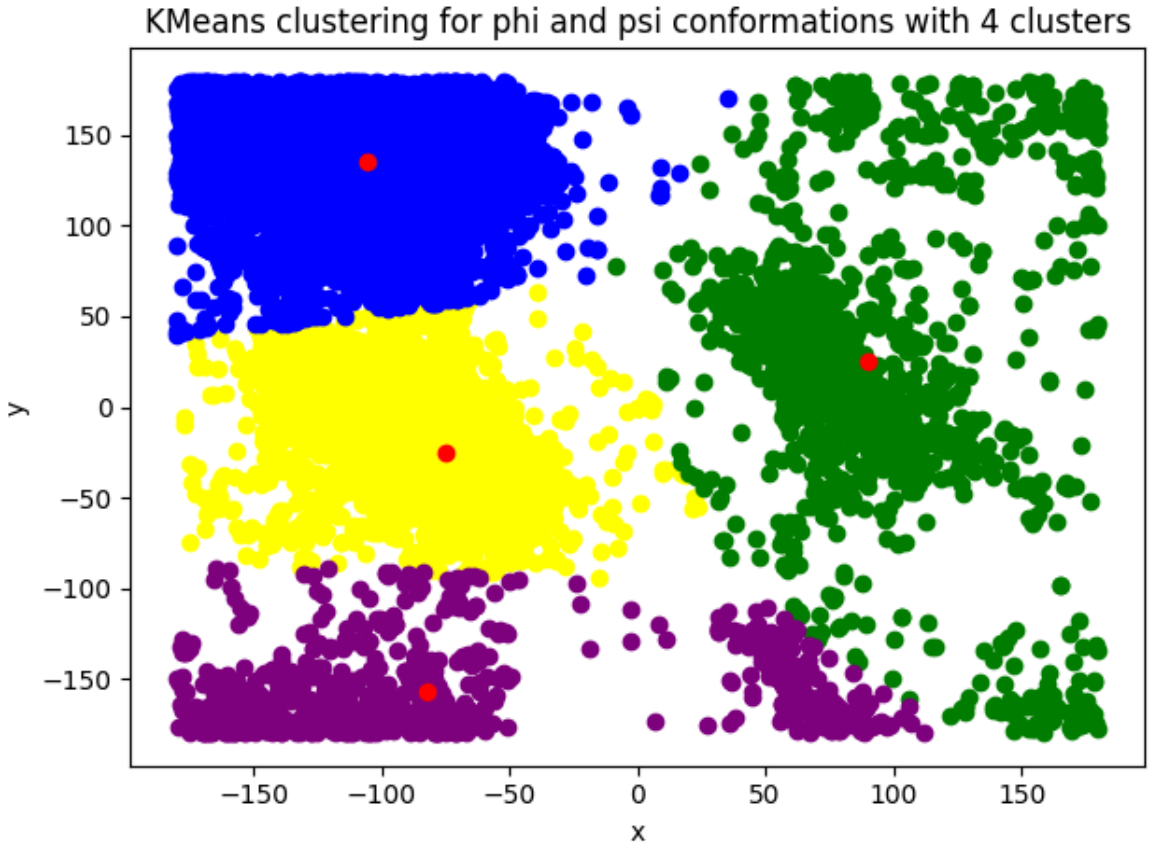
Based in the plot, we can assume that 3 is a good number of clusters, because after that, the inertia is not becoming signifficantly smaller.

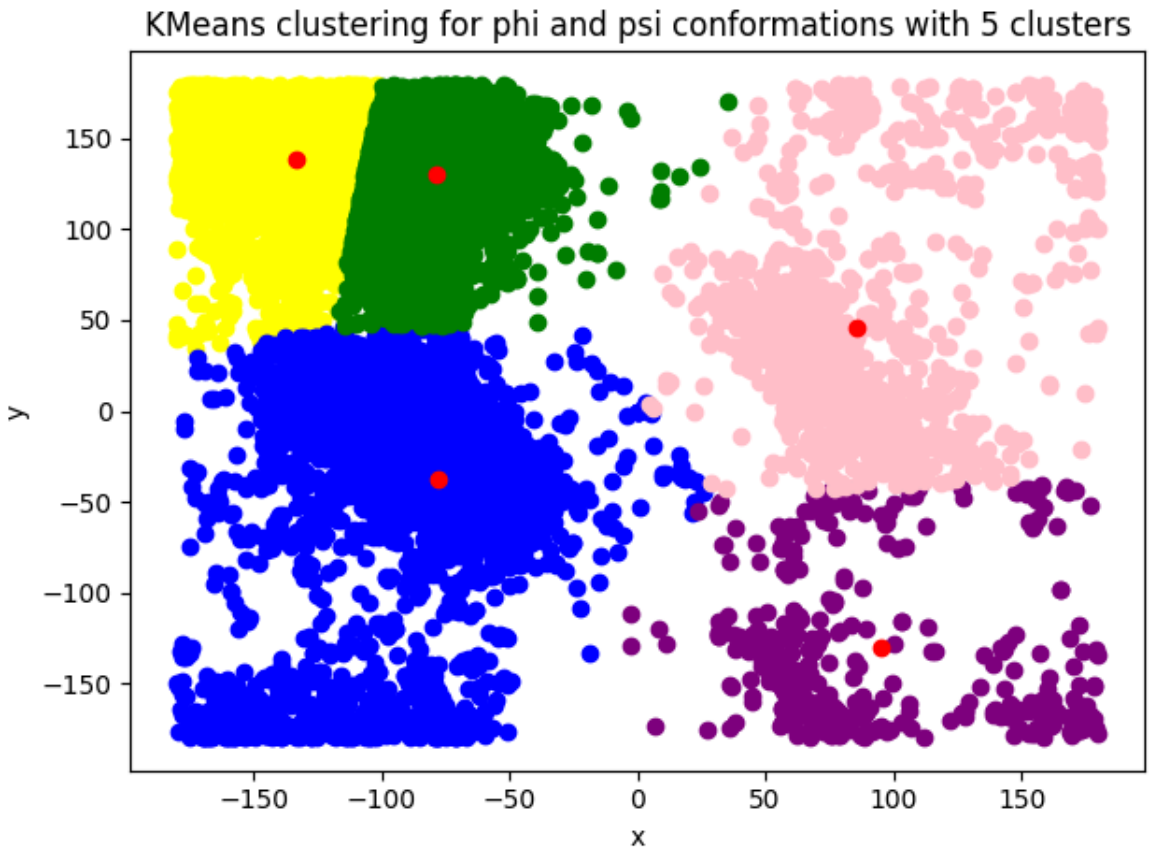
In [9]:
```
clustering_proteins_df_kmeans = np.array(clustering_proteins_df)

colors = ['yellow', 'blue', 'green', 'purple', 'pink', 'brown', 'black', 'cyan',

for num_clusters in range(1, 10):
    km = KMeans(n_clusters=num_clusters)
    label = km.fit_predict(clustering_proteins_df_kmeans)

    for curr_label in range(num_clusters):
        filtered_label = clustering_proteins_df_kmeans[label == curr_label]
        plt.scatter(filtered_label[:,0] , filtered_label[:,1] , color = colors[c

    centroids = km.cluster_centers_
    plt.scatter(centroids[:,0] , centroids[:,1], color='red')
    plt.title(f'KMeans clustering for phi and psi conformations with {num_cluste
    plt.xlabel('x')
    plt.ylabel('y')
    plt.tight_layout()
    plt.show()
```

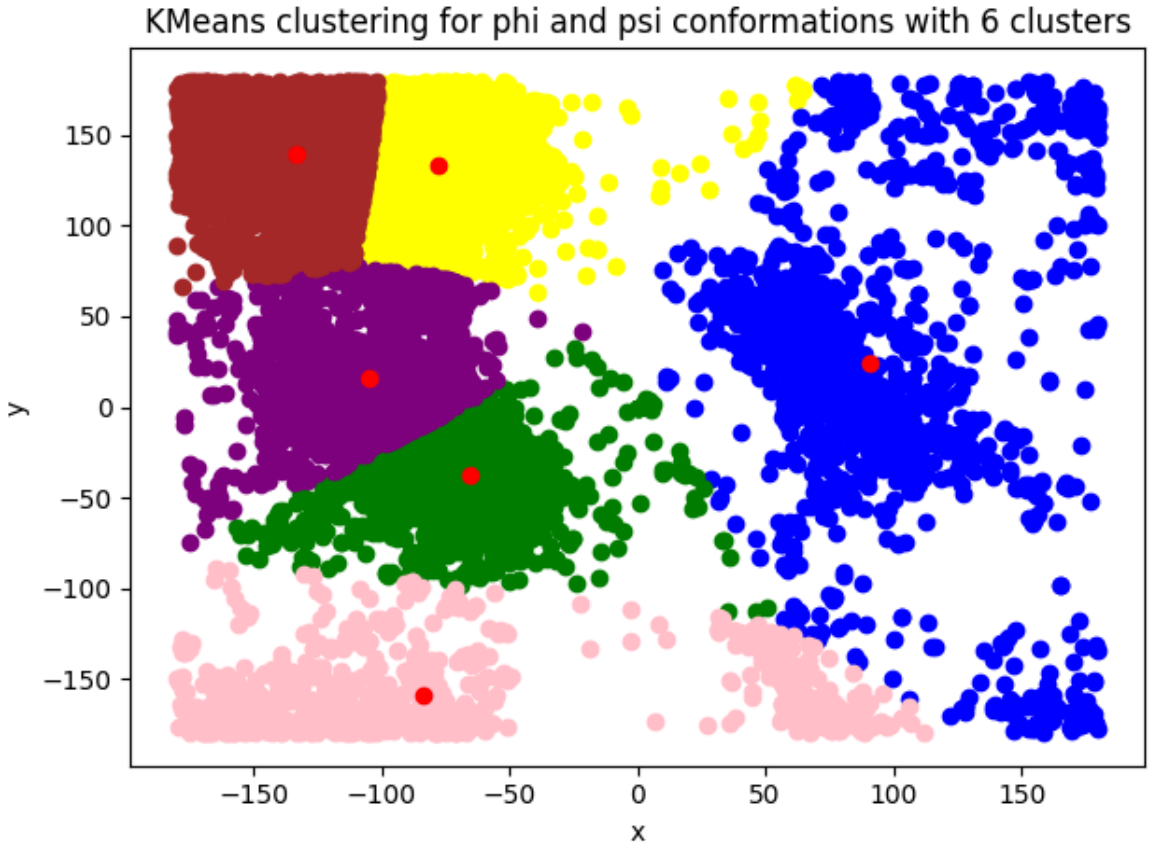

KMeans clustering for phi and psi conformations with 1 clusters
150
100
50
0
−50
−100
−150
y
−150
−100
−50
0
50
100
150
x

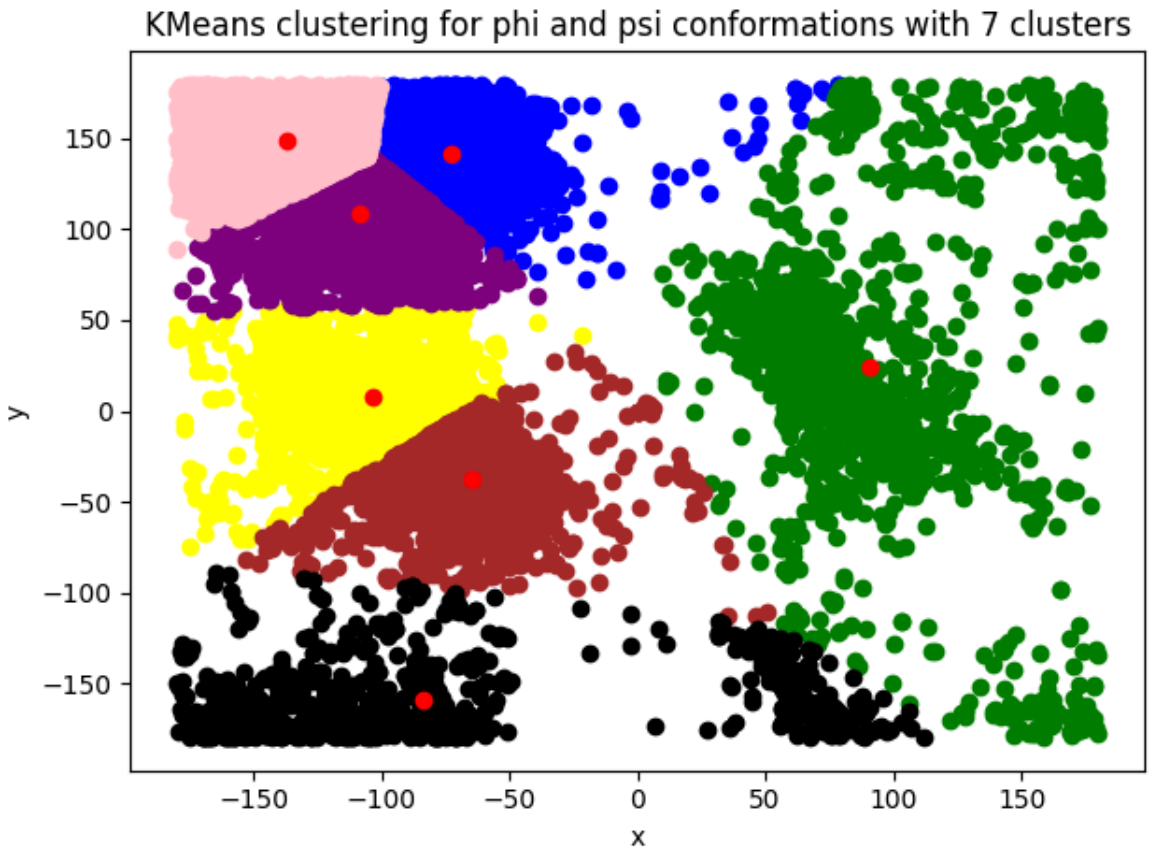

KMeans clustering for phi and psi conformations with 2 clusters
150
100
50
0
−50
−100
−150
y
−150
−100
−50
0
50
100
150
x

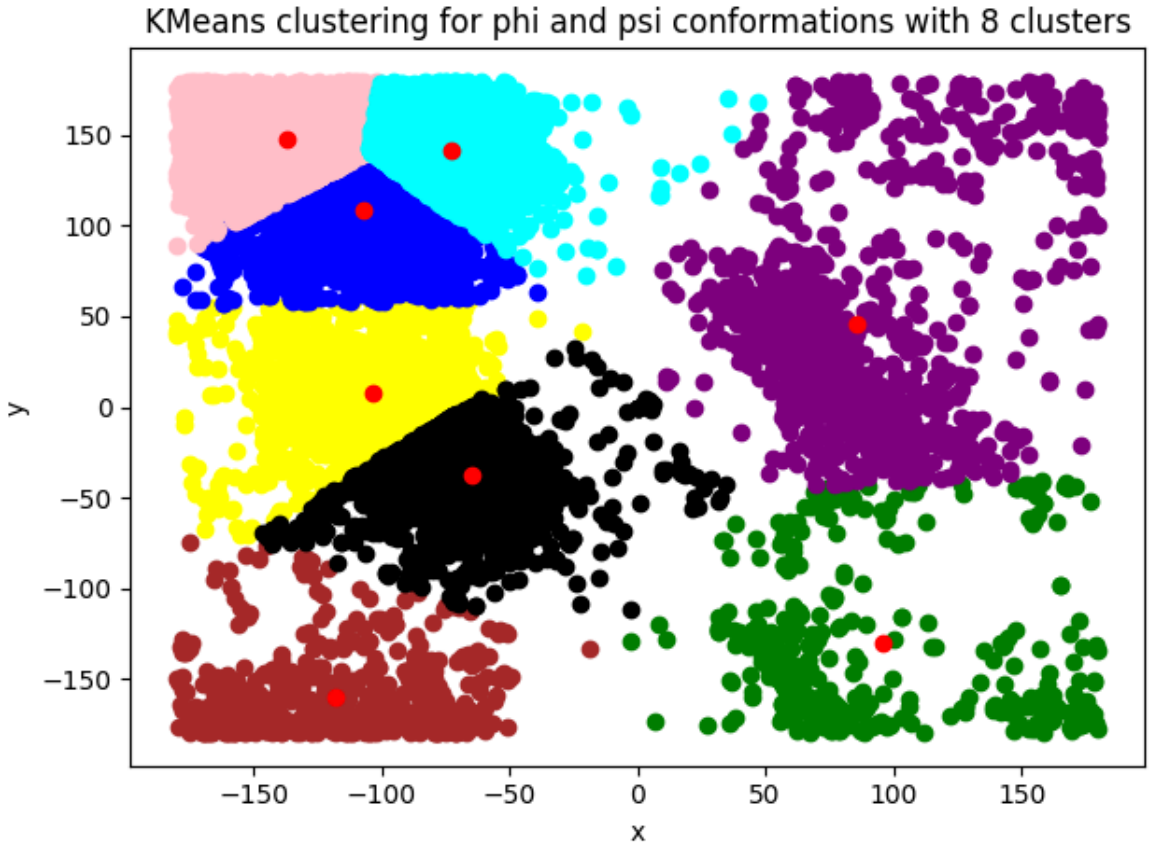

KMeans clustering for phi and psi conformations with 3 clusters
150
100
50
0
−50
−100
−150
y
−150
−100
−50
0
50
100
150
x

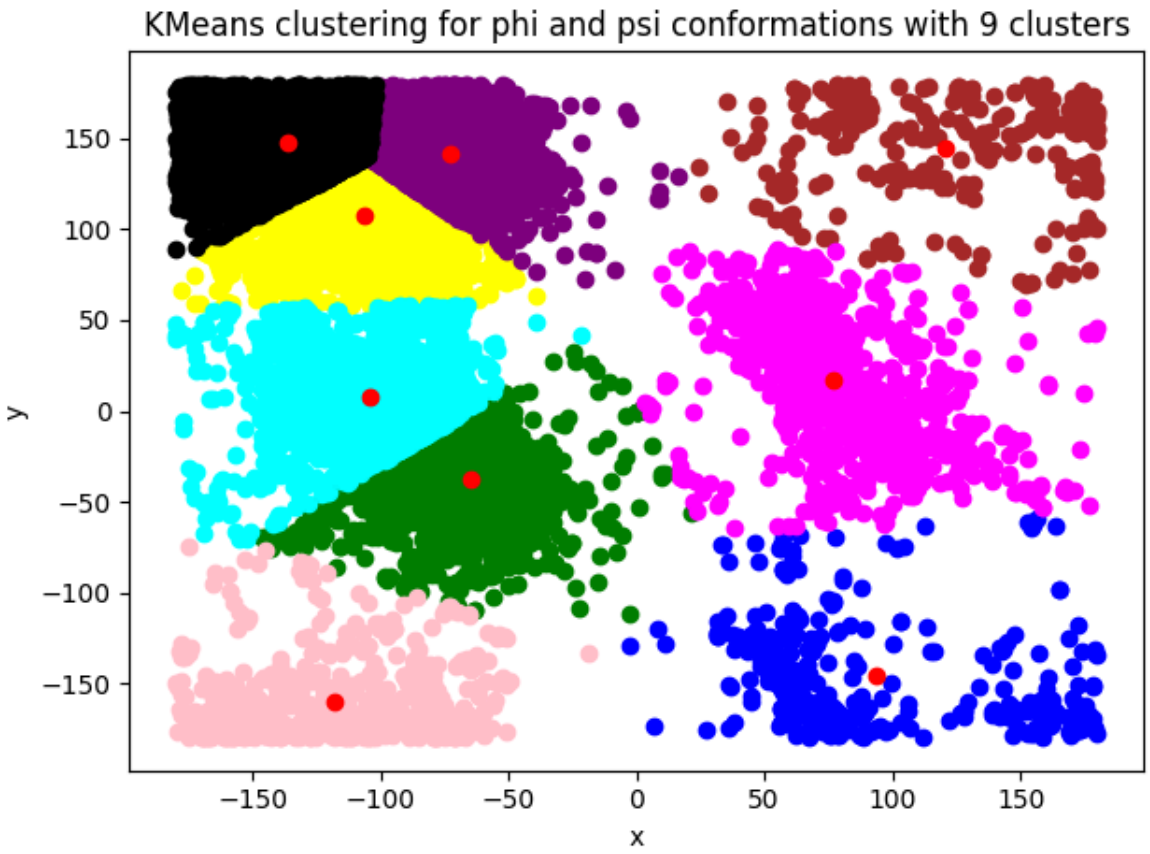

KMeans clustering for phi and psi conformations with 4 clusters
150
100
50
0
−50
−100
−150
y
−150
−100
−50
0
50
100
150
x


KMeans clustering for phi and psi conformations with 5 clusters
150
100
50
0
−50
−100
−150
y
−150
−100
−50
0
50
100
150
x


KMeans clustering for phi and psi conformations with 6 clusters
150
100
50
0
−50
−100
−150
y
−150
−100
−50
0
50
100
150
x


KMeans clustering for phi and psi conformations with 7 clusters
150
100
50
0
−50
−100
−150
y
−150
−100
−50
0
50
100
150
x


KMeans clustering for phi and psi conformations with 8 clusters
150
100
50
0
−50
−100
−150
y
−150
−100
−50
0
50
100
150
x



a. Experiment with different values of K. Suggest an appropriate value of K for this task and motivate this choice.

Based on the K means algorithm and also on the inertia plot, we can conclude that the best number of clusters should be 3. Unlike the assumption from the scatter plot, some of the border areas look like a separate cluster but K means is deciding them better as part of other cluster.

b. Do the clusters found in part (a) seem reasonable?

Based on the visualization of both scatter plot and 2D histogram, 3 clusters sound reasonable since K means is selecting them based on the centroids distance. So, yes, the prediction, based only on the scatter plot seems to be very accurate and it is a good initial direction for further research.

3. Use the DBSCAN method to cluster the phi and psi angle combinations in the data file.

In [10]:
```
clustering_proteins_df_dbscan = proteins_df[['phi', 'psi']].values
clustering_proteins_df_dbscan
```

Out[10]:
```
array([[-149.312855,  142.657714],
       [ -44.28321 ,  136.002076],
       [-119.972621, -168.705263],
       ...,
       [-113.586448,  112.09197 ],
       [-100.668779,  -12.102821],
       [-169.95124 ,   94.23368 ]])
```

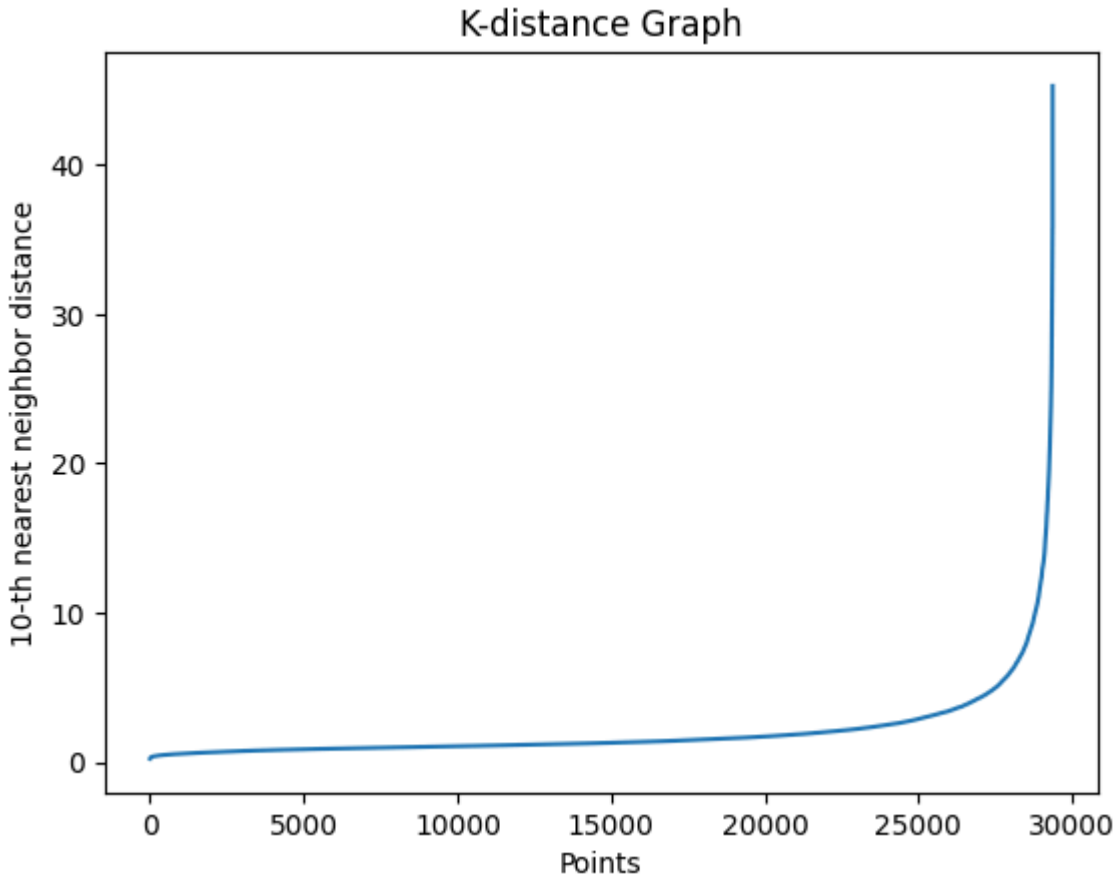
a. Motivate the choice of:

i. the minimum number of samples in the neighbourhood for a point to be considered as a core point, and

ii. the maximum distance between two samples belonging to the same neighbourhood ("eps" or "epsilon").

1. the minimum number of samples in the neighbourhood for a point to be considered as a core point - we will choose it as the number_of_features * 2
2. epsilon should be the maximum distance between two samples belonging to the same neighbourhood. Let's now choose the epsilon value:

In [11]:
```
neighbour = NearestNeighbors(n_neighbors=10)
neighbour.fit(clustering_proteins_df_dbscan)
distances, _ = neighbour.kneighbors(clustering_proteins_df_dbscan)
distances = np.sort(distances[:, 9])

plt.plot(distances)
plt.xlabel('Points')
plt.ylabel(f'10-th nearest neighbor distance')

plt.title('K-distance Graph')
plt.show()
```

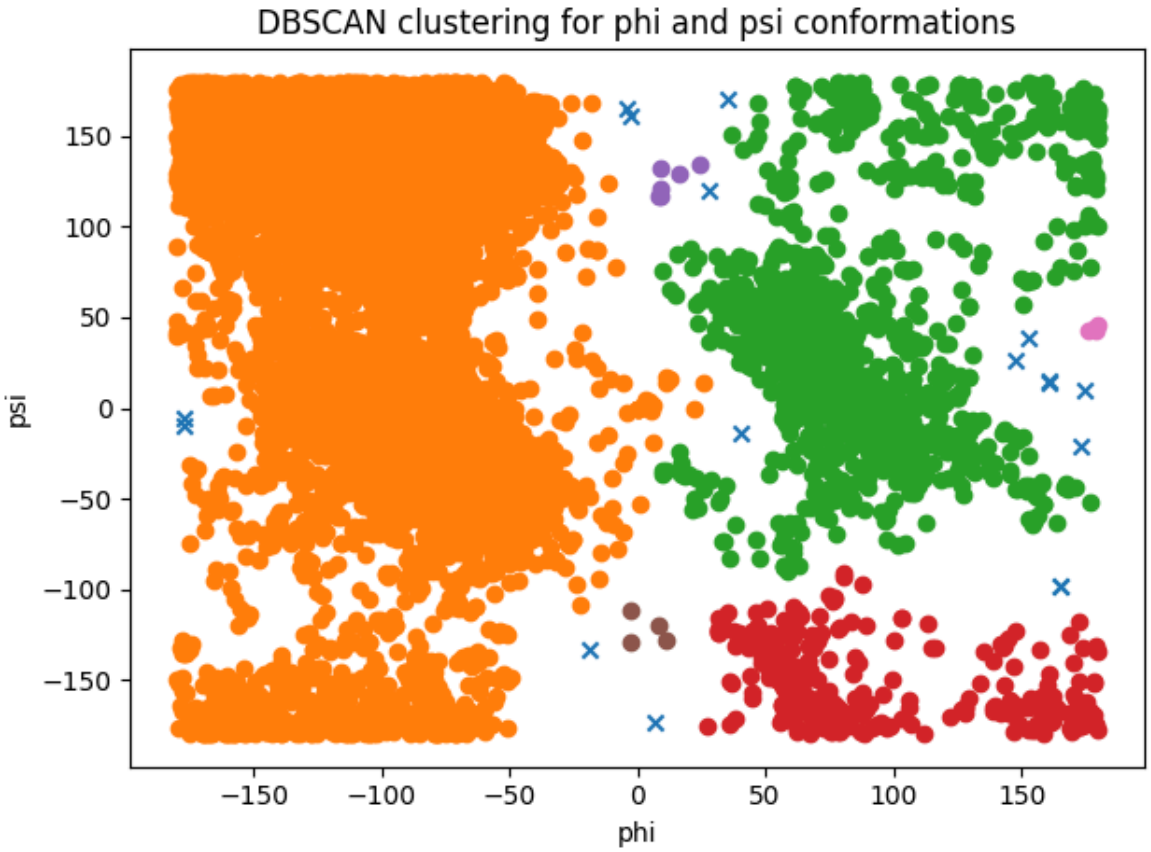


In [12]:
```
epsilon = np.percentile(distances, 90)
epsilon
```

Out[12]: 3.7215470442338865

3.7 would be the best epsilon value if the data is standartized. We will check the differences between the data being standartized and not.

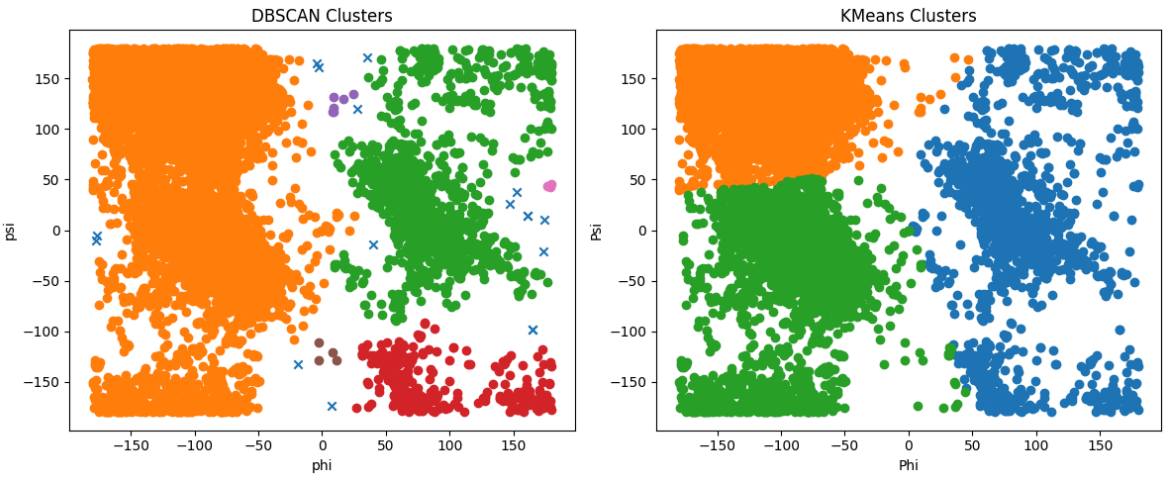
In [26]:
```
epsilon = 15
min_num_samples = 4

db = DBSCAN(eps=epsilon, min_samples=min_num_samples)
db_labels = db.fit_predict(clustering_proteins_df_dbscan)
proteins_df["dbscan_cluster"] = db_labels

unique_labels = np.unique(db_labels)

for label in unique_labels:
    subset = proteins_df[proteins_df["dbscan_cluster"] == label]
    if label == -1:
        plt.scatter(subset["phi"], subset["psi"], marker="x")
    else:
        plt.scatter(subset["phi"], subset["psi"])

plt.xlabel("phi")
plt.ylabel("psi")
plt.title("DBSCAN clustering for phi and psi conformations")

plt.tight_layout()
plt.show()
```



Compare the clusters found by DBSCAN with those found using K-means.

In [14]:
```
kmeans = KMeans(n_clusters=3)
proteins_df["kmeans_cluster"] = kmeans.fit_predict(clustering_proteins_df_dbscan
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

ax = axes[0]
unique = np.unique(db_labels)

for label in unique:
    subset = proteins_df[proteins_df["dbscan_cluster"] == label]
    if label == -1:
        ax.scatter(subset["phi"], subset["psi"], marker="x")
    else:
        ax.scatter(subset["phi"], subset["psi"])

ax.set_title("DBSCAN Clusters")
ax.set_xlabel("phi")
ax.set_ylabel("psi")

ax = axes[1]
unique_km = np.unique(proteins_df["kmeans_cluster"])

for label in unique_km:
    subset = proteins_df[proteins_df["kmeans_cluster"] == label]
    ax.scatter(subset["phi"], subset["psi"])

ax.set_title("KMeans Clusters")
ax.set_xlabel("Phi")
ax.set_ylabel("Psi")

plt.tight_layout()
plt.show()
```



From my point of view, DBSCAN algorithm is clustering them more accurately, based on the visible results.

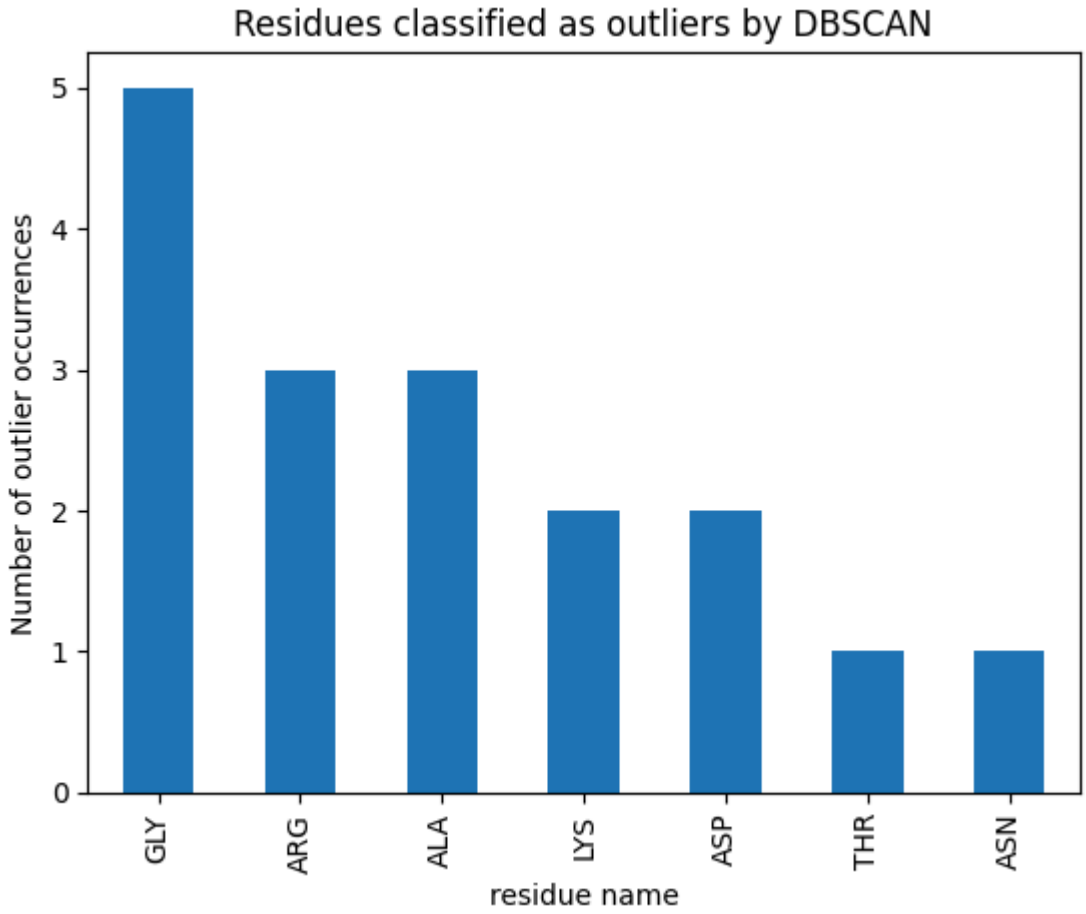
b. Highlight the clusters found using DBSCAN and any outliers in a scatter plot

c. How many outliers are found? Plot a bar chart to show how often each of the amino acid residue types are outliers

In [15]:
```
outliers = proteins_df[proteins_df["dbscan_cluster"] == -1]
n_outliers = len(outliers)
print("Number of outliers:", n_outliers)
```

```
counts = outliers["residue name"].value_counts()

counts.plot(kind="bar")
plt.ylabel("Number of outlier occurrences")
plt.title("Residues classified as outliers by DBSCAN")
plt.show()
```

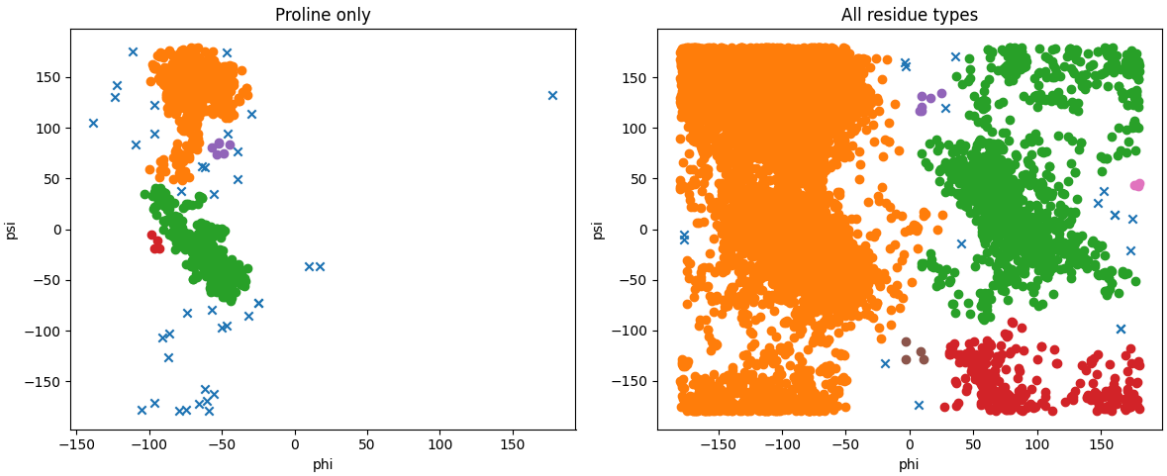
```
Number of outliers: 17
```



4. The data file can be stratified by amino acid residue type. Use DBSCAN to cluster the data that have residue type PRO. Investigate how the clusters found for amino acid residues of type PRO differ from the general clusters (i.e., the clusters that you get from DBSCAN with mixed residue types in question 3). Note: the parameters might have to be adjusted from those used in question 3

In [27]:
```
proline_df = proteins_df[proteins_df["residue name"] == "PRO"].copy()
X_proline = proline_df[["phi", "psi"]].values

proline_eps = 8
proline_min_samples = 3

db_pro = DBSCAN(eps=proline_eps, min_samples=proline_min_samples)
pro_labels = db_pro.fit_predict(X_proline)

proline_df["pro_cluster"] = pro_labels

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

ax = axes[0]
unique_pro = np.unique(pro_labels)
```

```
for label in unique_pro:
    subset = proline_df[proline_df["pro_cluster"] == label]
    if label == -1:
        ax.scatter(subset["phi"], subset["psi"], marker="x")
    else:
        ax.scatter(subset["phi"], subset["psi"])

ax.set_title("Proline only")
ax.set_xlabel("phi")
ax.set_ylabel("psi")


ax = axes[1]
unique_all = np.unique(proteins_df["dbscan_cluster"])

for label in unique_all:
    subset = proteins_df[proteins_df["dbscan_cluster"] == label]
    if label == -1:
        ax.scatter(subset["phi"], subset["psi"], marker="x")
    else:
        ax.scatter(subset["phi"], subset["psi"])

ax.set_title("All residue types")
ax.set_xlabel("phi")
ax.set_ylabel("psi")

plt.tight_layout()
plt.show()
```



We can conclude that the Proline residue type clusters are around (-110, -40) degrees. Phi values are narrow, psi are wider but still not that spread. Based on Ramachandran Principle, for proline there are limited B conformations and Proline produces little ammount of α-helical clusters.